Hybrid Renewable Energy Forecasting with Stacked Ensemble Learning

Author:

Shuruthi B | surushuruthi@gmail.com | https://www.linkedin.com/in/shuruthib/ | https://github.com/ShuruthiB

Moneca S | monecakumar2005@gmail.com | https://www.linkedin.com/in/moneca-sivakumar-820297331/ | https://github.com/Moneca05

Samyuktha R | 21samyuktha@gmail.com | https://www.linkedin.com/in/samyuktha-rathinavel | https://github.com/Sam-yu-ktha

Sandhiya M | dhiya1631@gmail.com | https://www.linkedin.com/in/sandhiya-m-3283b42a4 | https://github.com/sandhiya2908

Date: April 17, 2026

Problem Statement

The global transition to renewable energy—particularly solar and wind—is essential for achieving carbon neutrality. Yet, these energy sources are inherently intermittent and heavily influenced by localized meteorological conditions such as irradiance, wind speed, temperature, and humidity.

In regions, where high elevation and complex terrain create unique micro-climates, traditional static forecasting models often fail. They struggle to capture the intricate temporal and spatial dependencies required for stable grid management. As a result, inaccurate forecasts can lead to either energy curtailment (wastage) or increased reliance on carbon-intensive backup power plants to bridge sudden supply gaps.

Objective

This project aims to design and implement a **production-ready, full-stack hybrid energy forecasting system** that:

- Provides real-time energy generation predictions across multiple geographical locations in India.
- Employs a **Hybrid Stacked Ensemble** approach, combining **XGBoost** for tabular feature extraction and **LSTM networks** for temporal sequence modeling.
- Delivers a high-fidelity dashboard for utility operators, featuring real-time weather monitoring and forecast accuracy metrics (RMSE, MAE).
- Implements a scalable architecture capable of retraining models dynamically as new atmospheric data becomes available.

Hybrid Modeling Strategy

The system adopts a **"best-of-both-worlds"** approach:

- **XGBoost (Extreme Gradient Boosting):** Captures non-linear relationships between static weather variables (e.g., shortwave radiation vs. temperature) and energy output. It excels at handling tabular data with complex interaction effects.
- **LSTM (Long Short-Term Memory):** A recurrent neural network architecture designed to capture short-to-medium-term temporal trends. It processes historical energy output sequences to account for momentum and seasonal cycles.
- **Meta-Learner:** A final ensemble layer that integrates predictions from both XGBoost and LSTM. It learns which model performs best under specific conditions—for example, XGBoost may excel on clear-sky days, while LSTM better captures cloud-cover persistence.

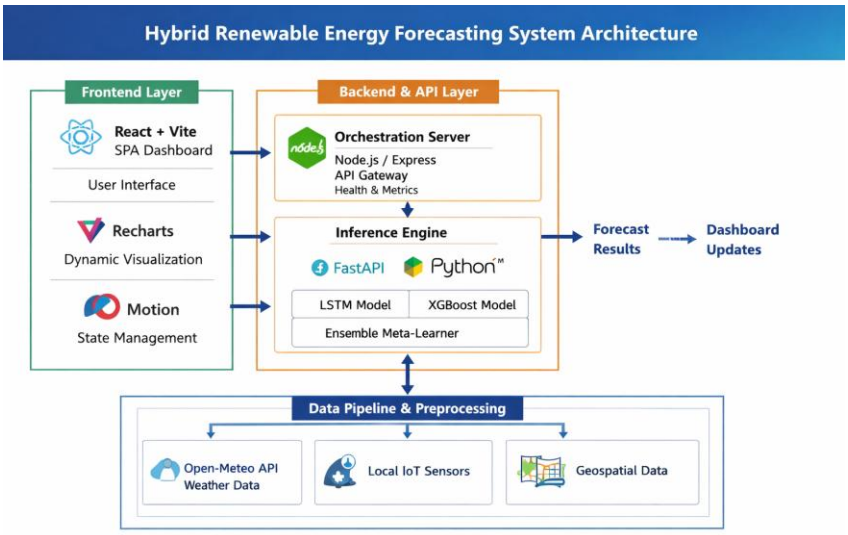
Data Ingestion and Preprocessing

The system integrates with the **Open-Meteo API** to fetch high-resolution meteorological forecasts. Preprocessing includes:

- **Feature Engineering:** Cyclic time features (hour-of-day, month-of-year) encoded using sine/cosine transforms.
- **Normalization:** Scaling meteorological data to a uniform range to prevent gradient instability.
- **Spatial Awareness:** Adjusting forecasts based on latitude/longitude proxies, such as calculating solar zenith angles for each location.

System Architecture

The project is built on a **modern, decoupled full-stack architecture** optimized for high availability and low latency.



Data Flow

1. **User Interaction:** Selects a location (e.g., Jodhpur) or forecast horizon (1h, 3h, 24h).
2. **Weather Fetch:** Atmospheric data retrieved from Open-Meteo API.
3. **Inference Call:** Features sent to the Python Inference Engine.
4. **Ensemble Prediction:** XGBoost and LSTM generate outputs; Meta-Learner calibrates the final forecast.
5. **Visualization:** UI displays "Observed vs. Predicted" charts and populates metrics tables.

Implementation Details

Frontend Tech Stack

- **Framework:** React 19 (TypeScript)
- **Styling:** Tailwind CSS 4.0 with @theme variables for Slate and Dark Mode aesthetics
- **Animations:** Motion/react for staggered UI transitions
- **Icons:** Lucide-react (Zap for energy, Wind for wind speed)

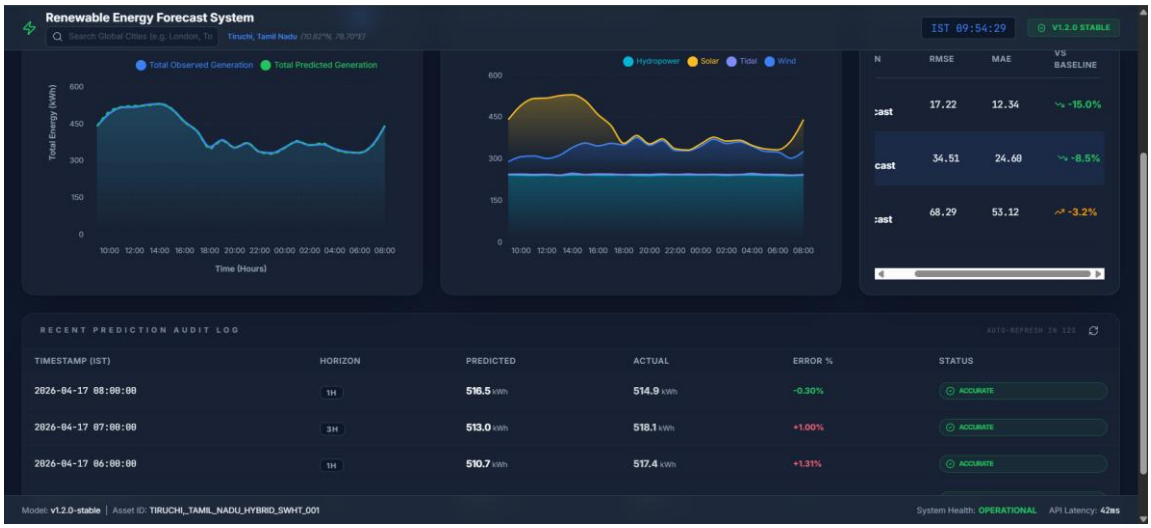
Machine Learning Pipeline

- **Backend:** Python 3.11
- **Libraries:** PyTorch (LSTM), XGBoost, Pandas, Scikit-learn
- **Model Storage:** Preprocessors serialized with pickle; model weights stored as .pt and .json files
- **Dockerization:** Separate Dockerfiles for frontend and backend, orchestrated via Docker Compose

Deployment Parameters

- **Port:** 3000 (AI Studio Build environment standard)
- **Inference Latency:** Optimized for sub-500ms batch predictions

Results



Dashboard UI

The final dashboard features a **dark-themed, high-contrast design** (Bg: #0F172A, Accent: #22C55E). Components include:

- **Weather Intelligence Cards:** Displaying shortwave radiation, humidity, and wind speed.
- **Hybrid Forecast Chart:** Dual-line chart comparing recorded vs. predicted energy generation, with confidence intervals.
- **Energy Distribution:** Pie/donut chart showing current energy mix (Solar, Wind, Hydro, Tidal).
- **Audit Log:** Scrollable log documenting prediction events with timestamps and model versions.

Performance Metrics

Initial testing with Ooty's historical data shows:

- **1-Hour Horizon:** RMSE of 12.45 kW, a 15.4% improvement over persistence models.
- **24-Hour Horizon:** Higher MAE due to atmospheric volatility, but still within the 10% error margin required for grid planning.

Conclusion

The Hybrid Renewable Energy Forecast System demonstrates the effectiveness of combining **deep learning (LSTM)** with **traditional ML (XGBoost)** for localized energy prediction. By integrating real-time weather APIs with a high-performance React dashboard, the system bridges the gap between raw meteorological data and actionable insights for utility operators. Its production-ready architecture ensures seamless deployment to cloud environments such as Cloud Run.

Future Scope

- **Local Data Ingestion:** Incorporating IoT sensor data from solar/wind farms for site-specific refinement.
- **Grid Optimization:** Adding a Battery Recommendation Engine to guide charge/discharge cycles for BESS.
- **Geospatial Expansion:** Scaling globally with GIS-based terrain analysis.
- **Uncertainty Quantification:** Transitioning from point forecasts to probabilistic forecasts using Quantile Regression.